

Excellent. Before ARINC 818-2, before Fibre Channel, before any protocol, we need to complete **Phase 0: Foundation Layer**.

I will approach this the same way I would train a new FPGA engineer in an aerospace or semiconductor company.

A warning:

Phase 0 is not about memorizing definitions.

Phase 0 is about building a mental model that will allow you to understand any digital communication protocol later.

Many engineers struggle with protocols because they have gaps in Phase 0.

PHASE 0 OBJECTIVE

At the end of Phase 0, you should be able to explain:

- How information becomes bits.
- How bits move through hardware.
- How hardware stores bits.
- How hardware synchronizes bits.
- How data is transmitted between devices.
- Why protocols exist.
- Why packet structures exist.
- Why clocking matters.
- Why FIFOs exist.
- Why CRC exists.
- Why serialization exists.
- Why high-speed links need encoding.

Without looking at any protocol specification.

PHASE 0 STRUCTURE

Module 0.1

Binary Information Theory

Module 0.2

Digital Logic Fundamentals

Module 0.3
Sequential Logic

Module 0.4
Clocking Fundamentals

Module 0.5
Data Movement

Module 0.6
Memory Structures

Module 0.7
Synchronization

Module 0.8
State Machines

Module 0.9
Data Integrity

Module 0.10
Communication Fundamentals

Module 0.11
Serialization

Module 0.12
Protocol Thinking

MODULE 0.1

Binary Information Theory

Most people skip this.

Big mistake.

Everything later depends on it.

What is information?

Suppose:

Temperature = 27°C

Human understands.

FPGA does not.

FPGA understands:

0

1

only.

Therefore:

27 decimal

=

11011 binary

Information must be represented.

Why binary?

Electronic circuits naturally create:

Low voltage

High voltage

Example:

0 V = Logic 0

1.8 V = Logic 1

or

0 V = Logic 0

3.3 V = Logic 1

depending on technology.

Bit

Smallest unit.

0
or
1

Nibble

4 bits

Example:

1010

Byte

8 bits

Example:

11001100

Word

Architecture dependent.

Examples:

16-bit
32-bit
64-bit
128-bit

Data Representation

Must understand:

Unsigned

00001010
=
10

Signed

Two's complement.

Understand:

- positive
 - negative
 - overflow
-

Fixed Point

Used heavily in FPGA.

Example:

Q8.8

Floating Point

Less common in protocol logic.

WHAT YOU MUST BE ABLE TO EXPLAIN

1. Why binary exists.
2. Why voltage represents data.
3. Difference between signed and unsigned.
4. Bit, byte, word.
5. Binary to decimal conversion.

6. Hexadecimal notation.

MODULE 0.2

Combinational Logic

Definition:

Output depends only on present input.

Example:

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

OR gate.

Must know:

NOT

AND

OR

XOR

NAND

NOR

XNOR

Understand:

Truth tables.

Not memorize.

Understand.

Boolean Algebra

Must know:

$$A + 0 = A$$

$$A \cdot 1 = A$$

$$A + A = A$$

$$A \cdot A = A$$

Reason:

Protocols often contain:

`crc_enable <= valid and ready;`

Need to understand instantly.

MODULE 0.3

Sequential Logic

Most protocols are sequential.

Not combinational.

Need storage.

Latch

Understand:

- level sensitive
 - transparency
-

Flip-Flop

Fundamental storage element.

Stores:

1 bit

Example:

```
if rising_edge(clk) then
  q <= d;
end if;
```

Must understand:

Setup Time

Data stable before clock.

Hold Time

Data stable after clock.

Clock-to-Q

Delay.

Violations lead to:

Metastability

Important.

Registers

Collection of FFs.

Example:

32-bit register

stores

32 bits

Protocols use registers everywhere.

MODULE 0.4

Clocking Fundamentals

This module alone causes many FPGA failures.

What is Clock?

Periodic timing reference.

Example:

100 MHz

means

100 million cycles/sec

Period:

$$T = 1/F$$

Example:

100 MHz

=

10 ns

Why clock?

Synchronizes operations.

Without clock:

Chaos.

Understand:

Rising Edge

Falling Edge

Duty Cycle

Jitter

Skew

Phase

PLL

Phase Locked Loop.

Used to:

- multiply clocks
 - divide clocks
-

MMCM

Xilinx clock manager.

Must know purpose.

Clock Domain

Critical concept.

Example:

100 MHz

and

125 MHz

are different domains.

Never assume signals can cross safely.

MODULE 0.5

Data Movement

Understand:

How data physically moves.

Example:

Sensor

→ FPGA

→ Memory

→ Display

Data movement creates:

- latency
 - throughput requirements
-

Latency

Time to travel.

Throughput

Amount per second.

Example:

1 Gbps

Bandwidth

Maximum throughput.

Must distinguish all three.

Interview favorite.

MODULE 0.6

Memory Structures

Protocols depend heavily on buffering.

Register

Small.

Fast.

Distributed RAM

Uses LUTs.

Block RAM

BRAM.

Large storage.

Need to understand:

- depth
- width

Example:

1024 x 32

FIFO

Most important memory structure.

Purpose:

Absorb timing differences.

Understand:

Write Pointer

Read Pointer

Full

Empty

Almost Full

Almost Empty

Protocols use FIFOs everywhere.

MODULE 0.7

Synchronization

One of the most important topics.

Suppose:

Signal generated
125 MHz

received by

100 MHz

logic.

Problem.

Metastability

FF cannot decide:

0?

or

1?

Result:

Unpredictable behavior.

Need understanding of:

2FF Synchronizer

Pulse Synchronizer

Handshake Synchronizer

Async FIFO

Without CDC knowledge:

High-speed protocols impossible.

MODULE 0.8

FSM

Everything becomes FSM.

Examples:

Protocol TX.

Protocol RX.

Packet Parser.

Link Training.

CRC Engine.

Types:

Moore

Mealy

Need ability to design:

IDLE
HEADER
PAYLOAD
CRC
DONE

state machine.

MODULE 0.9

Data Integrity

Why do protocols add extra bits?

Transmission errors.

Noise.

EMI.

Radiation.

Need understanding:

Parity

Detects simple errors.

Checksum

Stronger.

CRC

Industry standard.

Must understand:

- generator polynomial
- detection capability
- why CRC works

No need to derive math yet.

MODULE 0.10

Communication Fundamentals

Most important module before protocols.

Parallel Communication

Example:

32 wires

send

32 bits

simultaneously.

Advantages:

Fast.

Disadvantages:

Many wires.

Skew.

Routing difficulty.

Serial Communication

Example:

1 wire

send bits sequentially.

Advantages:

Few wires.

High speed.

Most modern protocols use serial.

Simplex

One direction.

Half Duplex

Both directions.

One at a time.

Full Duplex

Both directions simultaneously.

MODULE 0.11

Serialization

Critical for ARINC 818.

Suppose:

32-bit data

arrives.

Need serial transmission.

Serializer:

32 bits

→

1 bit stream

Deserializer:

1 bit stream

→

32 bits

Understand:

SERDES

CDR

Clock Recovery

Word Alignment

Comma Alignment

These are fundamental to all high-speed links.

MODULE 0.12

Protocol Thinking

This is the mindset module.

Never ask:

What does this field mean?

Ask:

Why does this field exist?

Never ask:

What is CRC?

Ask:

What problem is CRC solving?

Never ask:

Why FIFO?

Ask:

What failure happens if FIFO is removed?

Every protocol is ultimately:

Store Data



Format Data



Protect Data



Transmit Data



Recover Data



Verify Data



Deliver Data

Whether it's:

- UART
- SPI
- Ethernet
- PCIe
- Fibre Channel
- ARINC 818
- SpaceWire

The pattern is the same.

PHASE 0 COMPLETION CRITERIA

Before moving to ARINC 818, you should be able to comfortably explain and sketch:

Digital Logic

- Gates, Boolean algebra, combinational vs sequential logic

Storage

- Flip-flops, registers, RAM, BRAM, FIFOs

Timing

- Clock, setup, hold, jitter, skew, latency

Synchronization

- Metastability, CDC techniques, async FIFOs

Communication

- Serial vs parallel, simplex/half/full duplex

Integrity

- Parity, checksum, CRC

High-Speed Links

- SERDES, encoding, clock recovery, alignment

Architecture Thinking

- How data flows through an FPGA system

Only after these concepts are solid should we begin **Phase 1: Understanding ARINC 818-2 as a system**, because then every term in the specification will connect to concepts you already understand instead of appearing as isolated jargon.

For ARINC 818-2 work specifically, I would put extra emphasis on:

1. FIFOs and buffering
2. Clock-domain crossing
3. CRC concepts
4. Serial communication
5. SERDES/GT transceivers
6. Throughput and bandwidth calculations
7. FSM design

Those seven topics will appear repeatedly throughout the ARINC 818 implementation journey.

No.

What I gave earlier is a **Phase 0 roadmap**, not a complete Phase 0 mastery curriculum.

If your goal is:

"I should be able to learn ARINC 818-2, implement it in FPGA, debug it in hardware, and later learn any protocol independently."

Then there are still significant gaps.

A lot of training material teaches digital design. Very little teaches what an FPGA protocol engineer actually needs.

TRUE PHASE 0

Phase 0 should create five mental models:

Model 1 : How information exists

Model 2 : How hardware stores information

Model 3 : How hardware moves information

Model 4 : How hardware synchronizes information

Model 5 : How hardware verifies information

Everything else is derived from these.

GAP #1

Number Systems Mastery

Most graduates know binary conversion.

Not enough.

You need:

Binary

Understand:

10110110

without calculator.

Hexadecimal

Professional engineers think in hex.

Not binary.

Example:

FC Frame Header

0xBC

0x1E

0xA5

must immediately visualize bits.

Need instant conversion:

0xF

=

1111

0xA

=

1010

Bit Numbering

Understand:

Bit 0

Bit 1

Bit 2

...

Bit 31

Endianness

Critical.

Many protocol bugs occur here.

Need:

Big Endian

Little Endian

Byte Swapping

Word Swapping

When you see:

0x12345678

you should know:

How memory sees it.

How wire sees it.

How processor sees it.

These are different questions.

GAP #2

Timing Mathematics

Many FPGA engineers fail here.

Need ability to calculate instantly:

Frequency

100 MHz

Period

10 ns

Data Rate

Example:

64-bit bus

125 MHz

Bandwidth?

Need mental calculation:

$64 \times 125\text{M}$

=

8 Gbps

Latency

Need ability to calculate:

10 pipeline stages

100 MHz

=

100 ns

ARINC 818 discussions constantly require this.

GAP #3

Data Path Thinking

Most engineers see RTL.

Good engineers see data flow.

You must draw:

Input
↓
Buffer
↓
Formatter
↓
CRC
↓
FIFO
↓
Serializer
↓
Link

for every design.

Every protocol block eventually becomes:

Source

Buffer

Processor

Sink

Without this mindset protocol learning becomes impossible.

GAP #4

FPGA Resource Understanding

Earlier I mentioned LUTs.

Not enough.

Need deeper understanding.

LUT

What physically exists?

Why 6-input LUT?

Why synthesis creates LUT?

FF

What resource cost?

BRAM

Dual Port?

Simple Dual Port?

True Dual Port?

UltraRAM

When used?

DSP

When used?

When avoided?

Protocol engineers constantly estimate:

Can packet buffer fit?

Without FPGA resource awareness:

Architecture decisions become wrong.

GAP #5

FPGA Design Flow

Massive gap.

Before protocols:

Need complete flow.

Understand:

RTL

↓

Synthesis

↓

Implementation

↓

Bitstream

↓

Programming

Need purpose of:

Constraints

Timing Reports

Utilization Reports

DRC

Timing Closure

ARINC project success depends on this.

GAP #6

Reset Architecture

Fresh graduates underestimate this.

Senior engineers don't.

Need:

Synchronous Reset

Asynchronous Reset

Reset Synchronizer

Questions:

What happens when reset released?

What if reset crosses domains?

What if transceiver not ready?

Many protocol failures are reset issues.

GAP #7

CDC Deep Dive

Earlier only mentioned CDC.

Not enough.

Need complete mastery.

Understand:

Metastability Physics

Not just definition.

MTBF

Mean Time Between Failure.

Single Bit CDC

Multi-Bit CDC

Gray Code

Used in Async FIFOs.

Critical.

Async FIFO Architecture

Need full understanding.

Because:

Almost every protocol uses them.

GAP #8

FIFO Deep Dive

This deserves its own module.

Need:

Synchronous FIFO

Asynchronous FIFO

Understand:

Write Clock

Read Clock

Need:

Full

Empty

Overflow

Underflow

Backpressure

Every protocol implementation uses FIFOs.

GAP #9

FSM Engineering

Not FSM basics.

Engineering.

Need:

State Encoding

Binary

One-Hot

Gray

Need:

Recovery State

Illegal State Handling

Timeout State

Example:

ARINC RX parser.

Need robust FSM.

Not textbook FSM.

GAP #10

Error Detection Theory

Earlier only CRC introduction.

Need more.

Understand:

Single Bit Error

Burst Error

Random Error

Need:

Parity

Checksum

CRC

Comparison.

Need:

Why protocols choose CRC-16 vs CRC-32.

GAP #11

Memory Architecture

Need much deeper.

Understand:

Register File

BRAM

Dual-Port RAM

Ping-Pong Buffer

Circular Buffer

ARINC video systems heavily depend on buffering.

GAP #12

Hardware Debugging Foundations

This is a huge missing area.

Need:

Vivado ILA

Understand:

Trigger

Capture

Probe

Need:

How to debug:

Missing packet

CRC failure

FIFO overflow

Link loss

Before protocol study.

GAP #13

High-Speed Digital Fundamentals

Critical for ARINC.

Need understanding of:

Differential Signaling

LVDS

CML

Signal Integrity

Reflection

Termination

Impedance

Jitter

Eye Diagram

BER

Bit Error Rate

Without this:

GT transceivers become black magic.

GAP #14

Xilinx GT Transceivers

For ARINC this is mandatory.

Need understanding:

GTX

GTH

GTY

Need:

PCS

PMA

CDR

Elastic Buffer

Comma Detection

Most ARINC learning failures happen here.

GAP #15

Data Encoding

Before Fibre Channel.

Need:

NRZ

8b/10b

Why used?

Running disparity.

Control characters.

64b/66b

Why better?

These appear everywhere:

- Fibre Channel
 - Aurora
 - Ethernet
 - PCIe
-

GAP #16

Throughput Engineering

One of the most important professional skills.

Given:

1920×1080

60 FPS

24-bit RGB

Need ability to calculate:

Raw video bandwidth.

Need:

Payload efficiency.

Need:

Line overhead.

Need:

Protocol overhead.

ARINC work depends heavily on this.

GAP #17

Interface Fundamentals

Before protocol.

Need understanding of:

UART

SPI

I2C

AXI Stream

AXI Memory Mapped

Not because ARINC uses them.

Because they teach protocol thinking.

GAP #18

Verification Fundamentals

Huge gap.

Need:

Testbench

Stimulus

Monitor

Scoreboard

Assertions

Protocol work is 50% verification.

GAP #19

Documentation Reading Skill

This is rarely taught.

Yet crucial.

When opening a 500-page protocol spec:

Never read sequentially.

Learn:

1. Terminology
2. Architecture
3. Packet Structure
4. State Machines
5. Timing

in that order.

This is how senior engineers learn protocols.

GAP #20

System Thinking

The biggest gap.

Never see protocol alone.

Always ask:

Where does data come from?

Where does it go?

What if it arrives too fast?

What if it stops?

Who detects failure?

TRUE PHASE 0 COMPLETION CHECKLIST

If I were certifying you as ready for ARINC 818-2, I would expect you to comfortably explain and sketch from memory:

Mathematics

- Binary
- Hex
- Endianness
- Bandwidth calculations

Digital Logic

- Gates
- Boolean algebra
- Combinational logic

Sequential Logic

- FF
- Registers
- Setup/Hold
- Metastability

Timing

- Clocking
- PLL/MMCM
- Jitter
- Skew

FPGA Architecture

- LUT
- FF
- BRAM
- DSP
- Routing

Data Movement

- Pipelines
- Latency
- Throughput

Buffering

- FIFO
- Circular buffers
- Ping-pong buffers

CDC

- Synchronizers
- Async FIFO
- Gray code

FSM

- Design
- Recovery
- Timeout

Error Detection

- Parity
- Checksum
- CRC

Communication

- Serial

- Parallel
- Duplex modes

High-Speed Links

- SERDES
- CDR
- Encoding
- Eye diagrams

Xilinx Specific

- GT Transceivers
- Vivado Flow
- Timing Reports
- ILA

Verification

- Testbench
- Assertions
- Scoreboards

Engineering Thinking

- Data flow
- Resource estimation
- System integration
- Spec-reading methodology

Only after these areas are genuinely understood would I consider Phase 0 complete and move you to **Phase 1 (ARINC 818 System Understanding)**. For someone targeting aerospace FPGA protocol development, Phase 0 is often 60–70% of the battle; ARINC 818 becomes much easier once these foundations are solid.

No.

And this is an important realization:

A truly "zero-gap" Phase 0 is impossible to finish in a single document.

Because Phase 0 is not a list of topics. It is the complete prerequisite knowledge required to understand any digital communication protocol and implement it on an FPGA.

What we can do is define a **Phase 0 Knowledge Boundary**.

The question is:

"What must an FPGA engineer know before opening the ARINC 818-2 specification?"

Let's build that boundary rigorously.

The Ultimate Phase 0

Think of Phase 0 as constructing a building.

- Level 0A Mathematics
- Level 0B Computer Engineering Fundamentals
- Level 0C Digital Hardware Fundamentals
- Level 0D FPGA Fundamentals
- Level 0E Data Movement Fundamentals
- Level 0F Communication Fundamentals
- Level 0G High-Speed Communication Fundamentals
- Level 0H Verification Fundamentals
- Level 0I Debug Fundamentals
- Level 0J Engineering Thinking

Most graduates know parts of 0C.

Professional protocol engineers understand all ten.

LEVEL 0A

Mathematics Foundation

Most protocol engineers underestimate this.

Binary Arithmetic

Must perform mentally:

1010 + 0011

Hexadecimal Arithmetic

Need comfort with:

0xFF

0xBC

0x1E

Powers of Two

Instant recall:

2^8

2^{10}

2^{16}

2^{20}

2^{32}

Why?

Memory sizes.

FIFO depths.

Address calculations.

Protocol fields.

Logarithms

Need intuitive understanding.

Example:

1024 locations

requires

10 address bits

Because:

$$\log_2(1024)=10$$

Throughput Calculations

Need instant calculations.

Example:

$$1920 \times 1080 \times 60 \times 24$$

bits/sec.

No calculator dependency.

Percent Overhead

Critical for protocols.

Example:

$$8b/10b$$

overhead?

Need answer:

$$25\%$$

immediately.

LEVEL 0B

Computer Engineering Fundamentals

Often ignored.

Huge mistake.

Need understanding of:

CPU

What it does.

Memory

How storage works.

DMA

Direct Memory Access.

Interrupts

Cache

Basic understanding.

Reason:

Protocols rarely operate alone.

They interact with processors.

LEVEL 0C

Digital Hardware Fundamentals

The true foundation.

Voltage Levels

What is a logic 1?

What is a logic 0?

Noise margins.

Logic Families

Need awareness:

- TTL
 - CMOS
 - LVTTTL
 - LVCMOS
-

Gates

Not just symbols.

Need propagation delay understanding.

Combinational Logic

Need ability to identify:

$y \leq a \text{ and } b$;

as combinational.

Sequential Logic

Need ability to identify:

if rising_edge(clk)

as sequential.

Timing

Must deeply understand:

Setup

Hold

Recovery

Removal

Many engineers know only setup/hold.

Not enough.

LEVEL 0D

FPGA Fundamentals

This is where many software-style learners fail.

FPGA Fabric

Need mental picture.

Not abstract words.

Understand:

LUT

↓

FF

↓

Routing

↓

BRAM

↓

DSP

Configuration Memory

How FPGA becomes hardware.

Bitstream

What is it?

What isn't it?

Synthesis

Transforms RTL to gates.

Place and Route

Transforms logic into physical locations.

Timing Closure

What it means.

Why designs fail.

Constraints

Need understanding of:

`create_clock`

and timing intent.

Not syntax memorization.

LEVEL 0E

Data Movement Fundamentals

Most protocol engineers live here.

Need understanding of:

Pipeline

Why pipelining exists.

Latency

Not throughput.

Throughput

Not latency.

Bandwidth

Not throughput.

These are three different concepts.

Buffering

Need understanding of:

Register Buffer

FIFO Buffer

Frame Buffer

Circular Buffer

Ping-Pong Buffer

Flow Control

Why receivers must sometimes say:

STOP

to transmitters.

LEVEL 0F

Communication Fundamentals

Before any protocol.

What is communication?

Transfer of information.

Need understanding of:

Source

Channel

Destination

Communication Modes

Simplex

Half Duplex

Full Duplex

Parallel Transmission

Advantages.

Disadvantages.

Serial Transmission

Advantages.

Disadvantages.

Synchronization Methods

Synchronous

Asynchronous

This becomes critical later.

LEVEL 0G

High-Speed Communication Fundamentals

This is the hidden prerequisite for ARINC 818.

Why high-speed links are hard

Need understanding of:

Attenuation

Reflection

Crosstalk

EMI

Noise

Differential Signaling

Why two wires?

Impedance Matching

Conceptual understanding.

Termination

Purpose.

Jitter

Types:

Random

Deterministic

Eye Diagram

Must understand what:

- open eye
- closed eye

mean.

BER

Bit Error Rate.

Professional protocol engineers discuss BER routinely.

LEVEL 0H

Verification Fundamentals

A massive gap in many training programs.

Need understanding of:

Simulation

Purpose.

Limitations.

Testbench Architecture

Need components:

Stimulus

Driver

Monitor

Scoreboard

Assertions

Why they exist.

Functional Coverage

Purpose.

Protocols cannot be trusted without verification.

LEVEL 0I

Debug Fundamentals

Before protocol work.

Need debugging mindset.

Observe

Not assume.

Vivado ILA

Need understanding of:

Trigger

Capture

Window

Probe

Root Cause Analysis

Need ability to ask:

Where first did behavior become wrong?

Not:

Where did I notice it?

Huge difference.

LEVEL 0J

Engineering Thinking

This is the most important level.

Most courses never teach it.

Principle 1

Every protocol is a data movement problem.

Principle 2

Every protocol is a timing problem.

Principle 3

Every protocol is a synchronization problem.

Principle 4

Every protocol is an error detection problem.

Principle 5

Every protocol is a buffering problem.

Principle 6

Every protocol is a state machine problem.

Principle 7

Every protocol is a system problem.

Never study protocol in isolation.

The Missing ARINC-Specific Prerequisites

For ARINC 818 specifically, there is another mini-Phase-0 that many engineers miss.

Before reading ARINC 818 you should understand:

What is a video stream?

Pixel

Line

Frame

Resolution

Color depth

RGB

Frame rate

Active video

Blanking

Video timing

Video bandwidth calculations

Because ARINC 818 transports video.

Without video fundamentals, ARINC documentation becomes confusing.

The Final Readiness Test

Before Phase 1, I would expect you to answer all of these without notes:

Digital Design

- Why do flip-flops exist?
- What causes metastability?
- Why are FIFOs needed?

Timing

- Difference between latency and throughput?
- Difference between setup and hold?

FPGA

- What happens during synthesis?
- What happens during place and route?

Communication

- Why use serial links?
- Why use differential signaling?

Protocols

- Why do packets exist?
- Why do CRCs exist?
- Why do headers exist?

High-Speed Links

- Why is clock recovery needed?
- Why is encoding needed?

System Design

- What happens if receiver is slower than transmitter?

- What happens if clocks differ?

Video

- How much bandwidth is required for 1080p60 RGB video?

If you can answer all of those comfortably and draw the data path from memory, then **Phase 0 is sufficiently complete for beginning serious ARINC 818-2 study.**

The next step would not be ARINC yet. There is actually a **Phase 0.5 (Video Systems + Fibre Channel Foundations)** that should be completed before opening the ARINC 818-2 specification, because ARINC 818 is fundamentally **video transported over Fibre Channel technology**. That Phase 0.5 dramatically reduces the learning curve when we start Phase 1.

No.

And there is a deeper reason why I keep saying no.

When you say:

"No gaps allowed. Cover everything."

There are actually **three different meanings of Phase 0**:

1. Academic Phase 0 (Digital Electronics syllabus)
2. FPGA Engineer Phase 0 (RTL development readiness)
3. Protocol Engineer Phase 0 (Ready to learn and implement any protocol)

For ARINC 818-2 and future protocol work, we need the third one.

Even after the previous lists, there are still hidden knowledge areas that experienced engineers use unconsciously.

Let's identify the remaining gaps.

Meta-Gap #1: Abstraction Layers

Most engineers struggle with protocols because they don't understand abstraction.

You must learn to see the same data at multiple levels simultaneously.

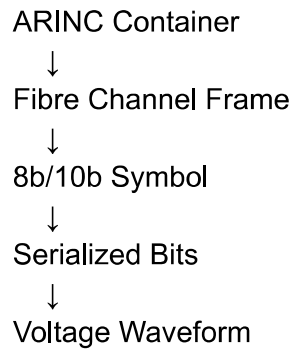
Example:

Video Pixel

↓

Video Word

↓



A protocol engineer must move between all these layers.

If you only understand one layer, debugging becomes impossible.

Meta-Gap #2: Representation vs Reality

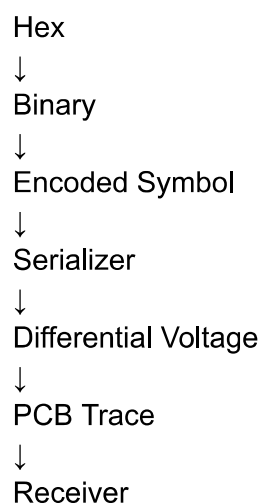
Example:

In RTL:

```
data <= x"BC";
```

Looks simple.

Reality:



You must always know:

"What physical phenomenon represents this logical value?"

Gap #21: Hardware Cost Awareness

Fresh graduates write logic.

Engineers estimate cost.

Need understanding of:

Area

- LUT usage
- FF usage
- BRAM usage

Performance

- Maximum frequency

Power

- Dynamic
- Static

Latency

Resource tradeoffs

Example:

One giant FIFO

vs

Several smaller FIFOs

Why choose one?

Gap #22: Scalability Thinking

Never ask:

Does it work?

Ask:

Does it still work at:

10 Mbps?

100 Mbps?

1 Gbps?

10 Gbps?

Protocols are often easy at low speed.

Hard at production speed.

Gap #23: Determinism

Critical in aerospace.

Need understanding of:

Deterministic latency

Non-deterministic latency

Worst-case latency

Best-case latency

Jitter budget

Many avionics protocols care more about determinism than speed.

Gap #24: Failure Analysis

Most training teaches success path.

Engineers study failure path.

For every block ask:

What if FIFO full?

What if CRC fails?

What if packet lost?

What if clock stops?

What if reset glitches?

This mindset is essential.

Gap #25: Requirements Engineering

Before design.

Need ability to read:

System Requirement

and derive:

RTL Requirement

Example:

Video latency < 1 ms

must become:

FIFO depth

Pipeline depth

Clock frequency

Gap #26: Interface Contracts

Protocols are contracts.

Need understanding of:

Producer obligations

Consumer obligations

Handshake rules

Error rules

Timing rules

This concept appears everywhere.

Gap #27: State Explosion

FSMs are simple in textbooks.

Real systems are not.

Need understanding of:

Normal states

Error states

Recovery states

Timeout states

Reset states

Every protocol implementation eventually becomes this.

Gap #28: Pipeline Thinking

Critical.

Need ability to identify:

Combinational path too long

and insert:

Pipeline stage

without breaking protocol behavior.

Gap #29: Data Width Conversion

Appears constantly.

Examples:

24-bit pixels

to

32-bit bus

or

64-bit AXI

to

32-bit protocol word

Need understanding of:

Packing

Unpacking

Alignment

Padding

Gap #30: Clock Architecture Thinking

Most juniors think:

One design
One clock

Reality:

GT clock
Video clock
AXI clock
Processor clock
Reference clock

Need ability to reason about multi-clock systems.

Gap #31: Bring-Up Methodology

Not usually taught.

Very important.

Need understanding of:

Clock?
↓
Reset?
↓
Link?
↓
Data?
↓
Protocol?

Debug in layers.

Never all at once.

Gap #32: Documentation Hierarchy

Before reading a protocol.

Need understanding of document types.

Standard

Defines protocol.

ICD

Defines implementation.

Datasheet

Defines device.

User Guide

Defines usage.

Errata

Defines known issues.

Many engineers confuse these.

Gap #33: Reading Hardware Diagrams

Must understand:

Block diagrams

Timing diagrams

State diagrams

Memory maps

Register maps

Data flow diagrams

Protocol documents use all of them.

Gap #34: Register Architecture

Protocols often expose configuration registers.

Need understanding of:

Read-only

Write-only

Read-write

Sticky bits

Status bits

Control bits

Gap #35: Interrupt Thinking

Even protocol cores often interact with processors.

Need understanding of:

Event



Interrupt



Software response

Gap #36: Packet Thinking

Before protocols.

Need mental model:

Every packet contains:

Identity

Control

Payload

Protection

This pattern appears almost everywhere.

Gap #37: Protocol Layering

Critical.

Need understanding of:

Application

Transport

Link

Physical

Not necessarily OSI details.

Need conceptual layering.

Gap #38: Hardware/Software Partitioning

Must ask:

Should this be:

RTL?

or

Software?

This decision appears constantly.

Gap #39: Reuse Thinking

Junior mindset:

Write code

Engineer mindset:

Build reusable block

Need understanding of:

Parameterization

Generic modules

Reusable FIFOs

Reusable CRC engines

Gap #40: Engineering Economics

Rarely taught.

Important.

Need awareness of:

Schedule

Complexity

Verification effort

Maintenance cost

Sometimes technically perfect solutions are rejected.

The Final Hidden Layer

The biggest missing piece.

Mental Compression

Senior engineers do not memorize protocols.

They compress them.

Example:

UART becomes:

Framing + Timing

SPI becomes:

Shift Register + Clock

Ethernet becomes:

Packet + MAC + PHY

ARINC 818 becomes:

Video + Fibre Channel

PCIe becomes:

Packets + Credit Flow Control + SERDES

You should strive to reduce every protocol into:

Data
↓
Format
↓
Buffer
↓
Protect
↓
Transmit
↓
Recover
↓
Verify
↓
Deliver

If you can do that, learning new protocols becomes dramatically easier.

Can Phase 0 Ever Be Truly Complete?

Strictly speaking, no.

Aerospace FPGA engineers with 20 years of experience still deepen their understanding of timing closure, signal integrity, CDC, transceivers, and verification.

So instead of "100% complete," use this criterion:

Phase 0 is complete when you can take an unfamiliar protocol document, identify the data path, timing path, buffering strategy, error-detection method, clocking architecture, and implementation blocks before reading the details.

That is the actual readiness threshold for moving into ARINC 818-2, Fibre Channel, PCIe, Ethernet, SpaceWire, Aurora, or almost any other FPGA protocol project.

For your specific ARINC 818 goal, after Phase 0 the next prerequisite I would teach is not ARINC itself. It would be:

Phase 0.5

- Video Systems Fundamentals
- Xilinx GT Transceivers
- Fibre Channel Fundamentals
- High-Speed Link Debug Methodology

because those four subjects account for a large fraction of the difficulty people encounter when learning ARINC 818.